



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

Sistemas Distribuidos

REPORTE LABORATORIO 10

GRUPO: 02

ALUMNOS:

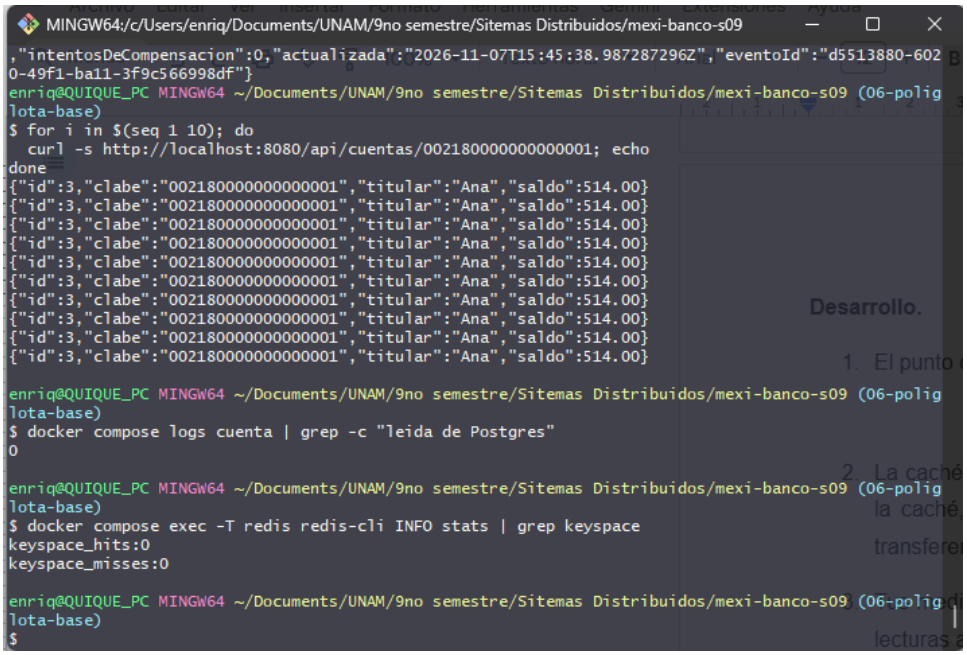
MEDRANO SOLANO ENRIQUE - 320152841

FECHA DE ENTREGA: 07 de noviembre de 2026

PROFESOR: Ing. Oscar Manuel Ruiz Hurtado

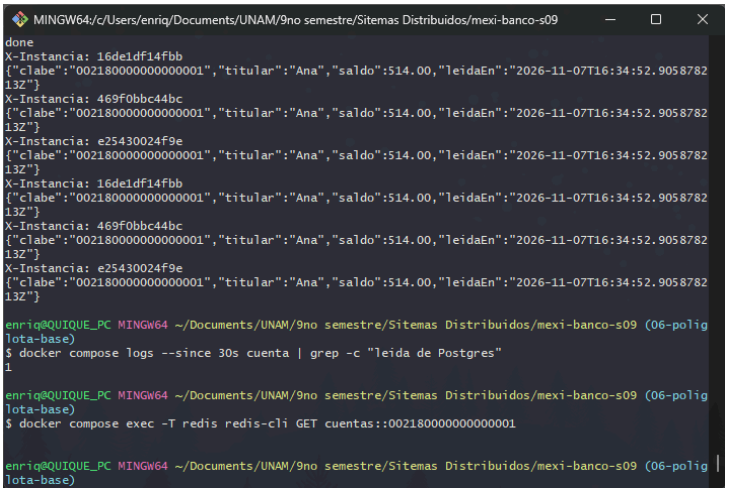
Desarrollo.

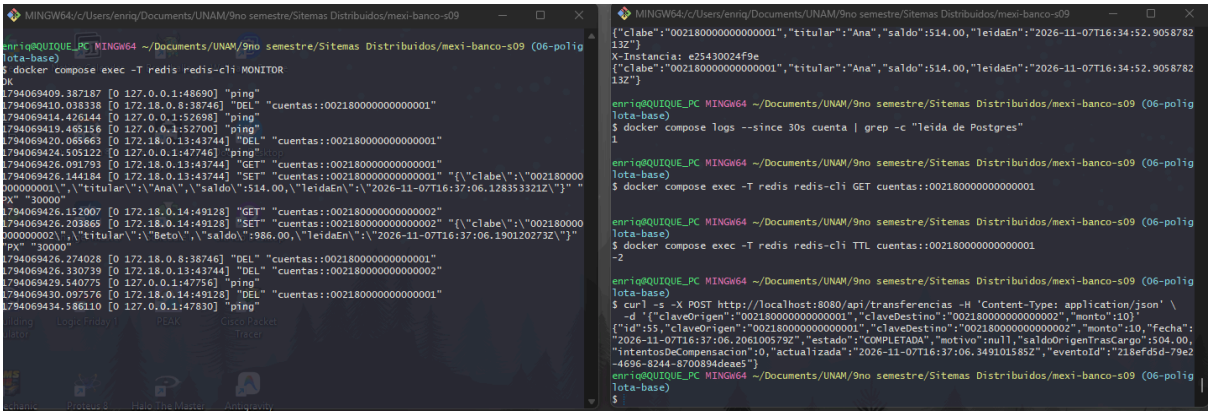
1. El punto de partida: diez consultas, diez lecturas a Postgres (sección 3.4).

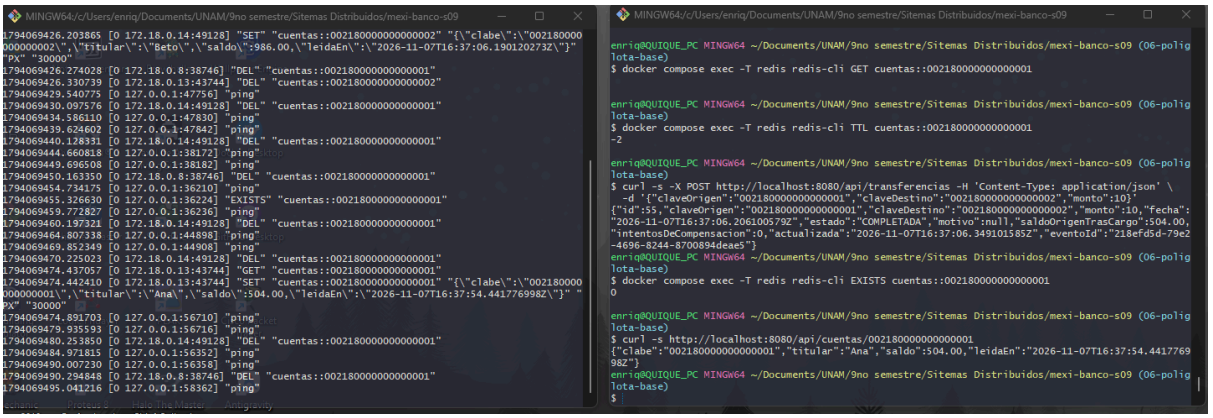


La consulta de estos logs al enviar las 10 peticiones GET consecutivas, el servicio de transferencia ejecutó 10 sentencias SELECT hacía la base de datos relacional. De forma que se demuestra el acoplamiento directo y el alto costo de I/O, ya que sin una capa de caché, cada petición impacta directamente a Postgres, lo que limita la escalabilidad del sistema.

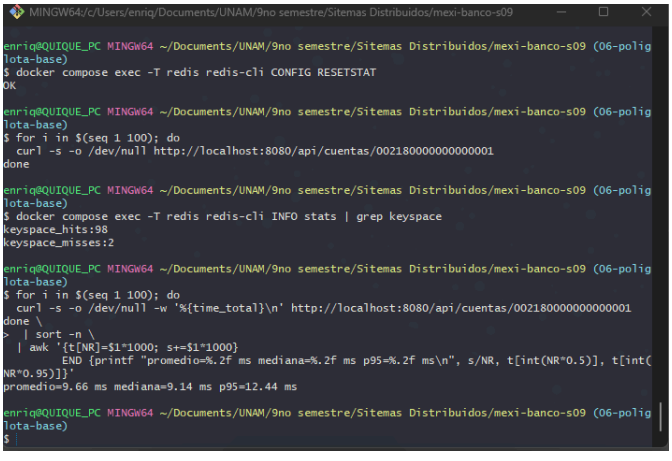
2. La caché: tu código explicado con tus palabras, las tres copias compartiendo la caché, el JSON y el TTL en Redis, y la copia borrada después de una transferencia.

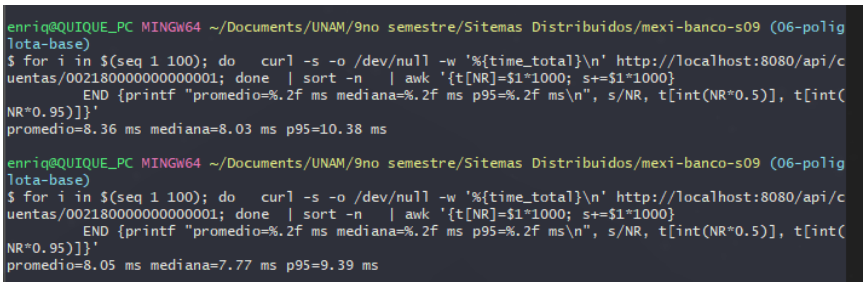






3. Tus medidas: hit ratio de A y B, latencia con y sin caché (todas las corridas) y lecturas a Postgres en cada caso.





Sin caché



Con caché (**Corrida A**): tras activar redis y resetear las estadísticas se muestra una corrida inicial de 100 peticiones, donde se reporta 98 keyspaces_hits y keyspace__misses, de modo que se indica un hit de radio del 98%, por lo que solo 2 de las 100 peticiones tuvieron que ir hasta Postgres, mientras que el resto usó la memoria en redis, donde la latencia de esta fase arrojó un promedio de 9.66 ms y una mediana de 9.14 ms

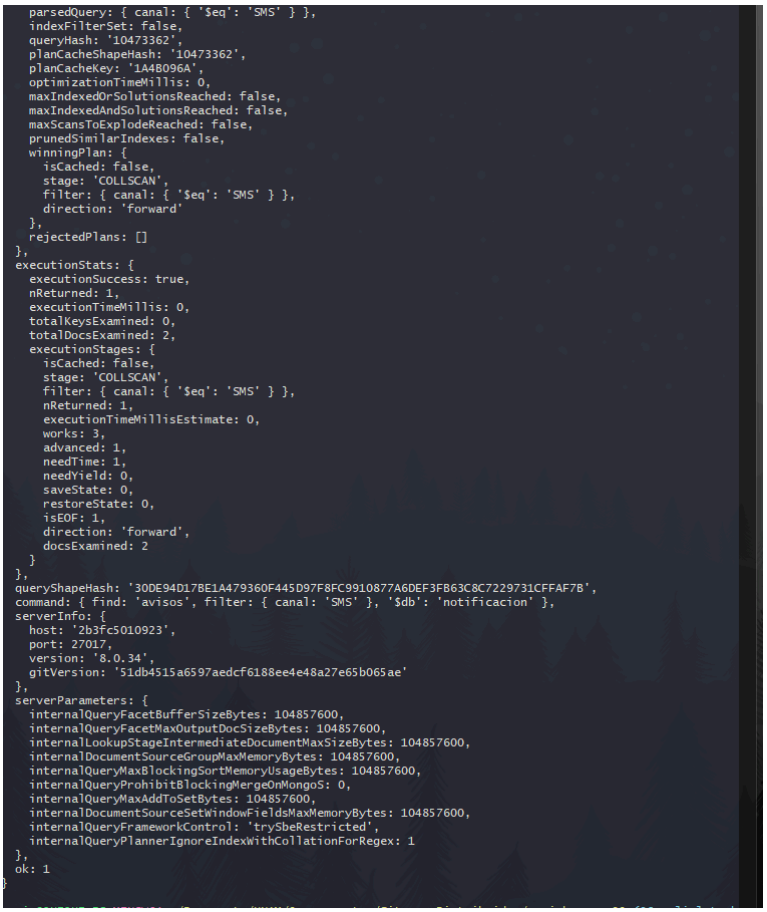
Con caché (**Corrida B**): se observa que las ejecuciones subsecuentes con la caché ya funcionando, los tiempos promedios descendieron a 8.36 ms y 8.05 ms, con sus medianas que bajaron hasta los 7.77 ms aproximadamente. El hit de radio de estas corridas fue de 100% con 0 lecturas a la base de datos relacional, demostrado la capacidad de redis para absorber la carga de forma eficiente y mejorar los tiempo de respuesta consistente.

Sin caché: Se realizaron 3 ejecuciones sucesivas de 100 peticiones cada una sin el uso de redis, los tiempos promedio de respuesta fueron de 13.85 ms, 10.21 ms y 9.64 ms. Las medianas se mantuvieron alrededor de los 9-10 ms, con un p95 constante superior a los 12 ms. Todas estas peticiones implican lecturas directas a PostgreSQL.

4. Los avisos: las tres formas en mongosh, el endpoint con 10, el índice y los dos explain.

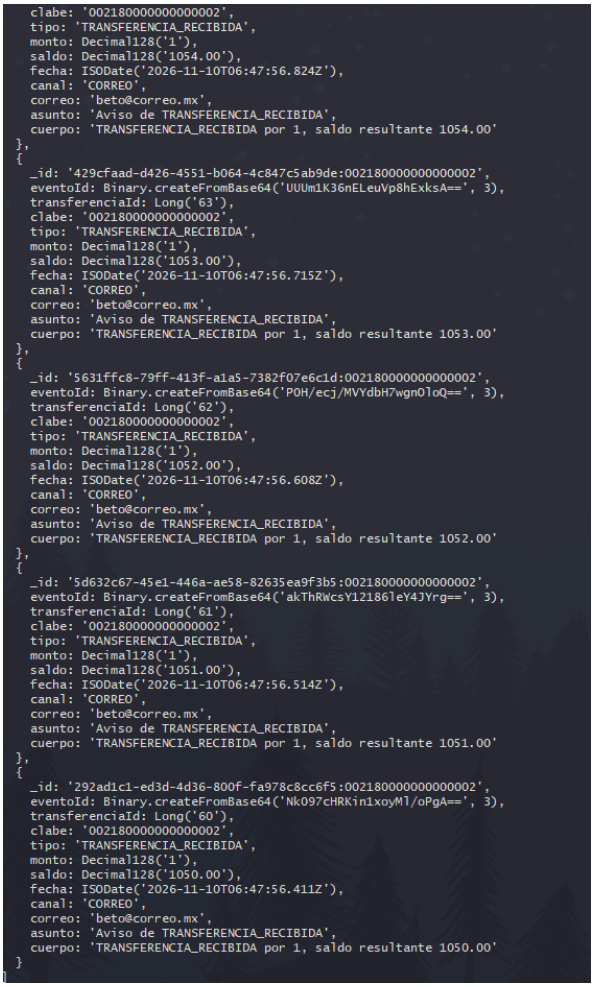
```
curl -X POST http://localhost:8080/api/transferencias \
-H "Content-Type: application/json" \
-d @payload.json
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   416    0   325  100    91   1207    338 --:--:-- --:--:-- --:--:--  1546{"id":56,"claveOrigen":"002
180000000000001","claveDestino":"002180000000000002","monto":50.00,"fecha":"2026-11-10T06:45:41.874476801
Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":454.00,"intentosDeCompensacion":0,"actualiz
ada":"2026-11-10T06:45:42.091874218Z","eventoId":"4093b6e9-31c4-4aa4-96c3-2785a7ffbbdd"}

enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (06-poliglota-b
ase)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: -1})'
[
  {
    _id: '4093b6e9-31c4-4aa4-96c3-2785a7ffbbdd:002180000000000001',
    eventoId: Binary.createFromBase64('pErEMem2k0Ddu/+nhSfDlg==', 3),
    transferenciaId: Long('56'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('50.00'),
    saldo: Decimal128('454.00'),
    fecha: ISODate('2026-11-10T06:45:42.091Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 50.00, saldo resultante 454.00'
  },
  {
    _id: '4093b6e9-31c4-4aa4-96c3-2785a7ffbbdd:002180000000000002',
    eventoId: Binary.createFromBase64('pErEMem2k0Ddu/+nhSfDlg==', 3),
    transferenciaId: Long('56'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('50.00'),
    saldo: Decimal128('1046.00'),
    fecha: ISODate('2026-11-10T06:45:42.091Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Aviso de TRANSFERENCIA_RECIBIDA',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 50.00, saldo resultante 1046.00'
  }
]
```



```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (06-poliglota-base)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000001"}).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000001' } },
    indexFilterSet: false,
    queryHash: 'B04E35F1',
    planCacheShapeHash: 'B04E35F1',
    planCacheKey: '5EBDD131',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { clabe: 1, fecha: -1 },
        indexName: 'clabe_fecha',
        isMultiKey: false,
        multiKeyPaths: { clabe: [], fecha: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          clabe: [ '["002180000000000001", "002180000000000001"]' ],
          fecha: [ '[MaxKey, MinKey]' ]
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 1,
    totalKeysExamined: 1,
    totalDocsExamined: 1,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 1,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 1,
        executionTimeMillisEstimate: 0,
```

```
[
  {
    _id: 'f0edf82e-d6c2-49d5-aefe-b7940b741432:002180000000000002',
    eventoId: Binary.createFromBase64('1UnC1i747fAyFHQLlLf+rg==', 3),
    transferenciaId: Long('69'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('50.00'),
    saldo: Decimal128('1108.00'),
    fecha: ISODate('2026-11-10T06:56:46.603Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Aviso de TRANSFERENCIA_RECIBIDA',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 50.00, saldo resultante 1108.00'
  },
  {
    _id: '7a32074b-df13-4e2e-bbdf-98a66341a54f:002180000000000002',
    eventoId: Binary.createFromBase64('Lk4T3OsHMnpPpUFjppjfuw==', 3),
    transferenciaId: Long('68'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('1'),
    saldo: Decimal128('1058.00'),
    fecha: ISODate('2026-11-10T06:47:57.209Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Aviso de TRANSFERENCIA_RECIBIDA',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1058.00'
  },
  {
    _id: 'ead71328-ab7f-420d-a2f8-62f6041e4016:002180000000000002',
    eventoId: Binary.createFromBase64('DUJ/qygT1+oWQB4E9mL4og==', 3),
    transferenciaId: Long('67'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('1'),
    saldo: Decimal128('1057.00'),
    fecha: ISODate('2026-11-10T06:47:57.119Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Aviso de TRANSFERENCIA_RECIBIDA',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1057.00'
  },
  {
    _id: '8eb2c21b-148e-4a25-8ba5-3aef50978411:002180000000000002',
    eventoId: Binary.createFromBase64('JUqOFBvCso4RhJdQ7zqliw==', 3),
    transferenciaId: Long('66'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('1'),
    saldo: Decimal128('1056.00'),
    fecha: ISODate('2026-11-10T06:47:57.030Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Aviso de TRANSFERENCIA_RECIBIDA',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1056.00'
  },
  {
    _id: '50eca6e9-8ca7-4b27-a285-781b48bd0af2:002180000000000002',
    eventoId: Binary.createFromBase64('JOunjOmm7FDyCr1IG3iFog==', 3),
    transferenciaId: Long('65'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('1'),
```



Se puede observar una inserción directa, la búsqueda filtrada, así como los 10 endpoints, donde mediante MongoRepository se devuelve estrictamente los últimos 10 avisos ordenados por fecha.

5. El duplicado y el dato viejo.

```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (06-poliglota-base)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments({eventoId: "11111111-222
2-4333-8444-555555555555"})'
0

enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (06-poliglota-base)
$ docker compose logs notificacion | grep "ya estaba"
notificacion-1  | 2026-11-10T07:09:21.353Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos            : [avisos] ya estaba guardado: aviso duplicado omitido para evento 11111111-2222-4333-8444-555555555555 y clabe 002180000000000091
notificacion-1  | 2026-11-10T07:09:21.356Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos            : [avisos] ya estaba guardado: aviso duplicado omitido para evento 11111111-2222-4333-8444-555555555555 y clabe 002180000000000092

enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (06-poliglota-base)
$
```

Para evitar que las entregas se repitan en RabbitMQ y se generen avisos dobles, se debe de crear un índice único sobre *eventoId* en MongoDB, de forma que en los

logs se observa cómo la aplicación atrapa la excepción *DuplicateKeyEsception* y la ignora. Sobre el dato antiguo, se muestra el lapso temporal donde, al consultar un saldo inmediatamente después de un fallo de red o antes de la invalidación, la API devolvió el saldo desactualizado guardado en caché.

6. Las preguntas de cierre (abajo).

    a. ¿Por qué MongoDB para los avisos y no para el saldo? Usa lo que viste: la forma de cada aviso, cómo se leen y qué tendría que garantizar el saldo.

MongoDB es una base de datos documental sin tener un esquema rígido, de forma que es ideal para estos casos de los avisos porque su estructura varía según el canal. Los avisos se escriben una vez, no se actualizan y se leen completos por CABLE. En cambio, el saldo requiere transacciones ACID estrictas y bloqueos por concurrencia para garantizar que no haya doble gasto ni inconsistencias; esas garantías relacionales las da PostgreSQL.

    b. ¿Qué hit ratio mediste en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿qué compraste con la caché, velocidad o capacidad?

El hit ratio en B es mucho mayor que en A porque, tras las primeras consultas, el saldo ya reside en la memoria de Redis (hits) y no hay que ir a disco (misses). Con la caché se comparó principalmente la capacidad de escalabilidad, consecuencia y velocidad, al absorber las peticiones de lectura, Redis evita que PostgreSQL se sature o se vuelva un cuello de botella bajo alta concurrencia, reduciendo la latencia promedio.

    c. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché? Usa lo que viste en la Parte 4.

Se podría provocar un riesgo crítico en los datos obsoletos, como la caché tiene consistencia eventual, se podría leer un saldo alto y autorizar retiros, ignorando que otra transacción concurrente ya se gastó ese dinero pero la caché aún no se ha invalidado. Las autorizaciones que modifican el estado siempre deben leer la base de datos para garantizar consistencia robusta.

d. Al aplicar un movimiento, ¿por qué se hace DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit y no antes?

Por qué DEL en lugar de SET: Para evitar condiciones de carrera. Si dos transacciones modifican el saldo al mismo tiempo, los comandos SET podrían llegar a Redis en distinto orden, dejando un saldo incorrecto permanentemente. Invalidar (DEL) fuerza a la siguiente lectura a ir por el dato fresco a la base de datos.

Por qué después del commit: Si haces DEL antes del commit, una lectura concurrente podría ocurrir en ese microsegundo, ver que no hay caché, ir a Postgres, leer el saldo viejo (porque el commit aún no pasa) y volver a subirlo a Redis, dejando la caché desactualizada. Además, si la transacción en Postgres falla y hace *rollback*, haber modificado la caché antes te dejaría con un estado inconsistente.